

Da função ao objeto: classes, atributos, métodos e encapsulamento

Como uma função vira método de uma classe, e por que proteger o estado interno de um objeto evita bugs silenciosos.

Faculdade Senac Pernambuco · Caruaru
Tecnologia em Análise e Desenvolvimento de Sistemas

Prof. Sebastião Rogério
22 de setembro de 2026

O que veremos hoje

BLOCO 1

Classes e objetos

O molde e a instância — como uma classe vira vários objetos diferentes, cada um com seu próprio estado.

BLOCO 2

Atributos e métodos

Dado e comportamento morando juntos — e como uma função da Aula 4 vira um método.

BLOCO 3

Encapsulamento e invariantes

Por que nem todo atributo deveria estar livre para qualquer alteração, e como o Python protege isso.

De onde viemos: laços, funções e módulos

PONTO 1

A comanda virou função

O que era um bloco de código repetido virou uma função com nome, parâmetro e retorno — testável com pytest.

PONTO 2

Argumento padrão

Uma função pode ter um valor pronto para quando quem chama não informa nada.

PONTO 3

Módulo reúne funções

Funções relacionadas do domínio da equipe foram organizadas num único arquivo, importável de qualquer lugar do projeto.

A Aula 4 resolveu como organizar comportamento. Faltou responder onde guardar o dado — assunto de hoje.

Quatro perguntas sobre o mesmo domínio

O domínio da equipe responde a uma pergunta diferente a cada aula.

AULA 2 · RESPONDIDO

Que estilo de código escrever?

Imperativo, funcional ou orientado a objetos — o mesmo relatório, três formas.

AULA 3 · RESPONDIDO

Como guardar o dado?

Variáveis, tipos corretos (Decimal para dinheiro) e coleções — listas, tuplas, dicionários.

AULA 4 · RESPONDIDO

Como organizar o comportamento?

Funções com retorno, argumento padrão, e módulos para reaproveitar entre arquivos.

HOJE · EM ABERTO

Como juntar dado e comportamento?

A classe como casa de ambos — e como proteger o dado de ser alterado de qualquer jeito.

Função sem dono do dado tende a repetir o mesmo `if` de validação em vários lugares. A classe resolve isso.

Primeiro se organiza comportamento em funções. Depois, dá-se a esse comportamento um dono — o objeto.

A função `calcular_total`, como ela era

```
DOMINIO/COMANDA.PY · AULA 4

1 def calcular_total(itens):
2     total = 0
3     for nome, preco in itens:
4         total += preco
5     return total
6
7 itens = [("água", 6), ("prato", 32)]
8 calcular_total(itens) # -> 38
```

CONFIRA Função pura: recebe a lista de itens por fora, não guarda nada sozinha.

A função ``calcular_total(itens)`` da Aula 4 funciona perfeitamente. Por que, mesmo assim, o time de POO diria que essa função "não tem dono"?

A resposta não é consertar a função — é perguntar quem deveria ser dono dos itens e do total. É essa pergunta que a aula responde.

BLOCO 1

Classes e objetos

O molde que define o que todo objeto vai ter, e a instância — o objeto concreto criado a partir dele.

O que é uma classe

Uma classe é o molde: define quais atributos e métodos todo objeto desse tipo vai ter. O objeto (instância) é a coisa concreta, criada a partir do molde.

NA LITERATURA

Python Software Foundation

"Classes provide a means of bundling data and functionality together." — é assim que a documentação oficial abre o capítulo sobre classes.

PYTHON SOFTWARE FOUNDATION. The Python Tutorial – Classes. docs.python.org.

Mesmo molde, estados diferentes

ANALOGIA

A planta de uma casa

A planta (classe) define os cômodos. Cada casa construída a partir dela (objeto) tem seus próprios móveis.

NO CÓDIGO

class NomeDaClasse:

Por convenção, o nome da classe começa com letra maiúscula — Comanda, e não comanda.

De função solta a classe: o `__init__`

```
DOMINIO/COMANDA.PY

1 class Comanda:
2     def __init__(self, mesa, itens=None):
3         self.mesa = mesa
4         self.itens = itens or []
5
6 # criando dois objetos a partir do mesmo molde
7 c1 = Comanda(mesa=5)
8 c2 = Comanda(mesa=12)
```

CONFIRA c1.mesa vale 5, c2.mesa vale 12 — mesmo molde, estados diferentes.

`__init__` e `self`

`__init__` é o método chamado automaticamente quando um objeto nasce. `self` é o próprio objeto sendo criado — todo método de instância recebe `self` como primeiro parâmetro.

NA LITERATURA

Mark Lutz, Learning Python

Lutz chama `__init__` de "método construtor": roda sozinho a cada criação de instância, sem precisar ser chamado à mão.

LUTZ, M. Learning Python. 5. ed. O'Reilly, 2013 — capítulo sobre classes.

`__init__` prepara o objeto

O QUE `__INIT__` FAZ

Prepara o objeto para uso

Recebe os dados iniciais (mesa, itens) e guarda cada um como atributo de self.

O ERRO COMUM

Esquecer o self

Sem `self.mesa = mesa`, o valor se perde assim que `__init__` termina — o objeto nasce vazio.

PAUSA PARA PENSAR

**Como em `calcular_total(itens)`, que revisamos:
no domínio da equipe, qual conjunto de
variáveis vocês sempre passavam juntas, de
função em função?**

Esse conjunto é forte candidato a virar os atributos da primeira classe da equipe.

Atributo de classe: compartilhado por todos

Atributo de classe é definido fora do `__init__`, direto no corpo da classe. Todos os objetos dessa classe compartilham o mesmo valor — diferente do atributo de instância, que cada objeto guarda por conta própria.

ATRIBUTO DE CLASSE

restaurante = "Sabor Caseiro"

Uma linha dentro da classe, fora do `__init__` — vale para toda Comanda criada, do mesmo jeito.

NA LITERATURA

Mark Lutz, Learning Python

Lutz separa os dois: atributo de classe vive no objeto-classe e é compartilhado; atributo de instância vive em cada objeto, isolado.

ATRIBUTO DE INSTÂNCIA

self.mesa

Cada objeto tem o seu — mudar em c1 não muda em c2.

LUTZ, M. Learning Python. 5. ed. O'Reilly, 2013 — atributos de classe versus atributos de instância.

Qual é a saída deste código?

EXERCÍCIO

```
1 class Comanda:
2     restaurante = "Sabor Caseiro"
3
4     def __init__(self, mesa):
5         self.mesa = mesa
6
7 c1 = Comanda(mesa=5)
8 c2 = Comanda(mesa=9)
9 print(c1.restaurante == c2.restaurante)
10 print(c1.mesa == c2.mesa)
```

True, depois False — por quê?



EXERCÍCIO · RESOLVIDO

```
1 c1 = Comanda(mesa=5)
2 c2 = Comanda(mesa=9)
3 print(c1.restaurante == c2.restaurante) # True
4 print(c1.mesa == c2.mesa)                # False
```

CONFIRA restaurante é atributo de classe — c1 e c2 compartilham o mesmo valor. mesa é atributo de instância — cada objeto tem o seu.

O que sustenta uma classe

- | | | |
|----|---|---|
| 01 | Molde e instância | A classe define a estrutura; cada objeto guarda seu próprio estado dentro dela. |
| 02 | <code>__init__</code> prepara o nascimento | É onde os dados iniciais viram atributos do objeto, via <code>self</code> . |
| 03 | <code>self</code> é o próprio objeto | Todo método de instância recebe <code>self</code> como primeiro parâmetro, sem exceção. |

Falta decidir o que o objeto sabe fazer com o dado que guarda — o próximo bloco.

BLOCO 2

Atributos e métodos

O dado que cada objeto carrega, e o comportamento que só faz sentido porque esse dado existe ali dentro.

Atributo: o dado que o objeto guarda

Atributo é uma variável associada a um objeto específico — cada instância tem sua própria cópia, independente das demais.

NA LITERATURA

Allen Downey, Think Python

Downey trata atributo como uma variável que pertence a um objeto específico — por isso o mesmo nome guarda valores diferentes em cada instância.

DOWNEY, A. Think Python. 2. ed. Green Tea Press, 2015 — capítulo sobre classes e objetos.

Cada objeto responde por si

ATRIBUTO DE INSTÂNCIA

self.mesa, self.itens

Definidos dentro de `__init__` (ou de outro método), pertencem a um objeto só.

ACESSO

objeto.atributo

`c1.mesa` lê o atributo `mesa` do objeto `c1` — cada objeto responde de forma independente.

De função a método: adicionar_item e total

```
DOMINIO/COMANDA.PY

1 class Comanda:
2     def __init__(self, mesa, itens=None):
3         self.mesa = mesa
4         self.itens = itens or []
5
6     def adicionar_item(self, nome, preco):
7         self.itens.append((nome, preco))
8
9     def total(self):
10         return sum(preco for _, preco in self.itens)
```

CONFIRA c1.adicionar_item("água", 6) muda só o estado de c1 — c2 nem percebe.

Método: comportamento que usa self

Método é uma função definida dentro da classe, que opera sobre o estado do próprio objeto — lê e altera atributos através de self.

NA LITERATURA

Luciano Ramalho, Python Fluente

Ramalho descreve método como uma função guardada como atributo de classe — por isso todo método de instância recebe self, o próprio objeto, de forma automática.

RAMALHO, L. Python Fluente. 2. ed. Novatec, 2022 — capítulo sobre um objeto pythônico.

Da Aula 4 para hoje

ANTES · AULA 4

calcular_total(itens)

Função solta — quem chama precisa sempre lembrar de passar a lista certa.

DEPOIS · HOJE

comanda.total()

Método — o objeto já sabe quais são seus itens, ninguém passa nada por fora.

PAUSA PARA PENSAR

Assim como `calcular_total(itens)` sempre recebia a mesma lista de itens: no domínio da equipe, qual função de lá recebia sempre os mesmos dados como parâmetro, aula após aula?

Essa é a função com maior chance de virar o primeiro método da classe de vocês hoje.

Vários métodos, um estado só

Dentro da mesma classe, métodos diferentes podem ler e alterar os mesmos atributos — é assim que eles colaboram sem precisar passar dado de um para o outro por parâmetro: todos enxergam o mesmo self.

MÉTODOS COLABORANDO

adicionar_item guarda, total lê

`adicionar_item` grava em `self.itens`; `total` lê o mesmo `self.itens` — sem receber nada por fora.

NA LITERATURA

Luciano Ramalho, Python Fluente

Para Ramalho, `self` é o que permite métodos da mesma classe colaborarem sem trocar parâmetro extra — todos enxergam o mesmo objeto.

SEM PASSAR NADA ENTRE ELES

Porque moram no mesmo objeto

Se fossem funções soltas, cada uma precisaria receber a lista por parâmetro.

RAMALHO, L. Python Fluente. 2. ed. Novatec, 2022 — métodos e estado compartilhado do objeto.

Qual é o valor de `c1.total()` ao final?

EXERCÍCIO

```
1 c1 = Comanda(mesa=5)
2 c1.adicionar_item("água", 6)
3 c1.adicionar_item("prato", 32)
4 c1.adicionar_item("sobremesa", 14)
5 print(c1.total())
```

52 — a soma dos três itens

EXERCÍCIO · RESOLVIDO

```
1 c1.adicionar_item("água", 6)
2 c1.adicionar_item("prato", 32)
3 c1.adicionar_item("sobremesa", 14)
4 print(c1.total()) # 6 + 32 + 14 = 52
```

CONFIRA Cada adicionar_item só altera self.itens de c1. total() soma tudo que está ali — 52.

O que muda de função para método

- | | | |
|----|----------------------------------|---|
| 01 | Atributo é dado com dono | Cada objeto guarda sua própria cópia — mudar um não afeta o outro. |
| 02 | Método é função com self | Já enxerga o estado do próprio objeto, sem precisar receber tudo por parâmetro. |
| 03 | Nada se perde na migração | A lógica da função da Aula 4 continua a mesma — só ganha um dono. |

Nada, até aqui, impede um total negativo. O problema do próximo bloco.

BLOCO 3

Encapsulamento e invariantes

Por que nem todo atributo deveria estar livre para qualquer valor — e como o Python protege isso sem esconder o objeto por completo.

O que é uma invariante

Invariante é uma regra que precisa ser verdadeira o tempo todo, em qualquer estado válido do objeto — por exemplo, "o preço de um item nunca é negativo".

NA LITERATURA

Bertrand Meyer, OOSC

Meyer formaliza o invariante de classe como a propriedade que toda instância válida precisa satisfazer sempre — base do "design by contract".

MEYER, B. Object-Oriented Software Construction. 2. ed. Prentice Hall, 1997 — conceito de invariante de classe.

O que acontece sem proteção

EXEMPLO DE INVARIANTE

Preço nunca negativo

Se a regra vale sempre, ela deveria ser garantida em um único lugar — não repetida em cada tela.

O RISCO DE NÃO PROTEGER

O bug que só aparece depois

O erro entra silenciosamente na criação do objeto, e só aparece quando o total já está errado.

Encapsulamento: um underscore de aviso

Encapsulamento é proteger o estado interno de um objeto, controlando como ele é lido e alterado. Em Python, um underscore no início do nome (`_itens`) é a convenção para "não mexa direto aqui de fora".

NA LITERATURA

Python Software Foundation

A documentação é direta: não existe forma de criar uma variável verdadeiramente privada em Python — o underscore é convenção, não trava.

PYTHON SOFTWARE FOUNDATION. The Python Tutorial – Private Variables. docs.python.org.

O underscore avisa, o método protege

CONVENÇÃO, NÃO TRANCA

_itens ainda é acessível

Python não impede o acesso — o underscore é um sinal para quem lê o código, não uma trava.

A TRAVA DE VERDADE

Fica no método

A proteção real acontece no método que altera o atributo, validando antes de aceitar.

Validando a invariante no ponto de entrada

```
DOMINIO/COMANDA.PY

1 class Comanda:
2     def __init__(self, mesa):
3         self.mesa = mesa
4         self._itens = []
5
6     @property
7     def total(self):
8         return sum(preco for _, preco in self._itens)
9
10    def adicionar_item(self, nome, preco):
11        if preco < 0:
12            raise ValueError("preço não pode ser negativo")
13        self._itens.append((nome, preco))
```

CONFIRA `comanda.adicionar_item("erro", -50)` agora levanta `ValueError` — o total nunca fica errado.

property: um método que parece atributo

@property transforma um método em algo lido como atributo, sem parênteses — `comanda.total`, não `comanda.total()`. Calcula um valor sob demanda, sem deixá-lo guardado (e desatualizado).

NA LITERATURA

Luciano Ramalho, Python Fluente

Ramalho chama property de "atributo calculado": parece um dado comum para quem lê, mas roda código de verdade por trás, a cada acesso.

RAMALHO, L. Python Fluente. 2. ed. Novatec, 2022 — capítulo sobre atributos e propriedades.

Com ou sem property

SEM PROPERTY

comanda.total()

Funciona, mas alguém pode tentar `comanda.total = 0` por engano, sem erro nenhum.

COM PROPERTY

comanda.total

Lê como atributo, sempre recalcula na hora — atribuir gera erro, a menos que exista um setter.

PAUSA PARA PENSAR

No domínio da equipe, qual atributo, se recebesse um valor absurdo (negativo, vazio, no passado), quebraria alguma regra de negócio?

Esse é o atributo que vale a pena proteger primeiro na prática de hoje.

Validar ou ignorar: duas estratégias

Ao encontrar um valor inválido, o método pode levantar uma exceção — interrompe a execução e avisa — ou ignorar/corrigir em silêncio — segue em frente sem contar a ninguém. A maioria dos times de POO prefere levantar exceção: uma falha visível é mais fácil de corrigir do que um bug silencioso.

LEVANTAR EXCEÇÃO

raise ValueError(...)

Interrompe na hora, aponta exatamente onde e por quê — o erro não se espalha.

IGNORAR EM SILÊNCIO

O dado errado nunca entra

Mas ninguém percebe o motivo — o bug aparece só depois, em outro lugar do código.

NA LITERATURA

Bertrand Meyer, OOSC

Meyer defende que violar uma invariante deve travar o programa na hora — um erro visível vale mais que um estado corrompido em silêncio.

MEYER, B. Object-Oriented Software Construction. 2. ed. Prentice Hall, 1997 — tratamento de violação de invariante.

O que acontece ao rodar a última linha?

EXERCÍCIO

```
1 comanda = Comanda(mesa=5)
2 comanda.adicionar_item("suco", 8)
3 comanda.adicionar_item("erro", -10)
```

Levanta ValueError, e o item não entra



EXERCÍCIO · RESOLVIDO

```
1 comanda.adicionar_item("suco", 8)
2 comanda.adicionar_item("erro", -10)
3 # ValueError: preço não pode ser negativo
```

CONFIRA A validação em `adicionar_item` barra o preço negativo antes de ele entrar em `self._itens` — o total nunca fica errado.

O que fica de hoje

Ponto	Regra prática
Classe e objeto	A classe é o molde; cada objeto guarda seu próprio estado.
Atributo e método	Atributo é dado com dono; método é função que já enxerga self.
Invariante	Uma regra que precisa valer sempre, em qualquer estado do objeto.
Encapsulamento	Underscore avisa; a proteção de verdade mora no método que valida antes de aceitar.

Uma classe bem desenhada garante que o dado nunca fica num estado que não devia existir.

Transformar o domínio da equipe em classe

- | | | |
|----|---|--|
| 01 | Escolher a entidade central | Do módulo da Aula 4, escolher a entidade mais usada do domínio da equipe (ex.: Pedido, Reserva, Aluno). |
| 02 | Criar a classe com <code>__init__</code> | Migrar as variáveis que sempre andavam juntas para atributos de self. |
| 03 | Migrar duas funções para métodos | Pegar duas funções da Aula 4 que operam sobre esses mesmos dados e reescrevê-las como métodos. |
| 04 | Proteger um atributo | Escolher um atributo com regra de negócio clara e validar seu valor no método que o altera (ou com <code>@property</code>). |
| 05 | Adaptar a suíte pytest | Ajustar os testes da Aula 4 para chamar métodos do objeto em vez de funções soltas — suíte precisa continuar verde. |

Fecha com git status, git add, commit no imperativo, push. Entregável: pacote de domínio com a primeira classe, métodos migrados e testes verdes.

Lista de exercícios

- | | | |
|----|--------------------------------------|---|
| 01 | Dois objetos, dois estados | Crie duas instâncias da classe do projeto de vocês e imprima o mesmo atributo de cada uma, provando que os estados são independentes. |
| 02 | Mais uma função vira método | Escolham outra função do domínio de vocês (fora as já migradas) e reescrevam como método, comparando a assinatura antes e depois. |
| 03 | Quebrem de propósito | Tentem atribuir um valor inválido a um atributo protegido e capturem o ValueError com try/except, imprimindo a mensagem. |
| 04 | Atributo de classe do projeto | Adicionem um atributo de classe (ex.: nome do sistema) e provem, com um print, que todos os objetos compartilham o mesmo valor. |
| 05 | Teste da invariante | Escrevam um teste pytest que garanta que um valor inválido levanta ValueError — não só que o valor válido funciona. |
| 06 | Property em ação | Transformem um cálculo do domínio de vocês em @property e mostrem, em comentário, a diferença de chamar com e sem parênteses. |

Cada exercício usa a classe que vocês já criaram na Atividade de hoje — não é um domínio novo, é o mesmo projeto, testado de mais ângulos.

REFERÊNCIAS

De onde veio cada afirmação desta aula

CLASSES E OBJETOS

PYTHON SOFTWARE FOUNDATION. The Python Tutorial — Classes. docs.python.org.

sintaxe de classe, `__init__` e `self`

DOWNEY, A. Think Python. 2. ed. Green Tea Press, 2015.

introdução a classes e objetos

LUTZ, M. Learning Python. 5. ed. O'Reilly, 2013.

`self` e métodos de instância

ENCAPSULAMENTO E INVARIANTES

MEYER, B. Object-Oriented Software Construction. 2. ed. Prentice Hall, 1997.

conceito de invariante de classe

RAMALHO, L. Python Fluente. 2. ed. Novatec, 2022.

property e atributos protegidos em Python

PRÓXIMO ENCONTRO · 29 DE SETEMBRO

AV1 (adiada) e POO III: herança, composição e polimorfismo

A AV1 marcada para hoje foi adiada para 29/09, junto com o fechamento do núcleo OO — herança só onde cabe, composição, e um método com várias implementações.

Para chegar pronto no dia 29

- Revisar classe, objeto, atributo, método, encapsulamento e invariante
- Trazer a classe migrada hoje, com pelo menos um atributo protegido e testes verdes
- A AV1 cobre as aulas 1 a 5 — revisar também paradigmas, tipos/coleções e funções/módulos